C++ 传址调用核心详解：从原理到实战

📌 C++ 传址调用（Call by Address）核心详解

传址调用是C++中**函数参数传递的核心方式之一**，本质是**将实参的内存地址作为参数传递给形参**，函数通过指针（地址）直接操作原实参的内存，而非创建副本。

一、核心概念：与传值调用的本质区别

这是理解传址调用的关键，先看对比表：

特性	传值调用（Call by Value）	传址调用（Call by Address）
传递内容	变量的**数据副本**	变量的**内存地址**
形参类型	普通变量（如 `int`）	指针类型（如 `int*`）
函数内修改影响实参？	❌ 完全不影响（仅改副本）	✅ 直接修改原实参的值
内存开销	随对象大小变化（拷贝大对象开销大）	固定开销（指针仅4/8字节）
典型场景	只读小对象，无需修改原数据	修改原数据、传递大对象（如数组/结构体）

二、代码示例：直观理解传址调用

最经典的「交换两个整数」例子，完美体现传值vs传址的差异：

1. 传值调用（错误，无法交换）

Code block

```
// 传值调用：形参是实参的副本，修改不影响原变量
void swap(int a, int b) {
    int temp = a;
    a = b;
```

```
    b = temp; // 仅修改副本a、b，原变量x、y完全不变
}

int main() {
    int x = 1, y = 2;
    swap(x, y); // 传x、y的副本，传值调用
    // 结果：x=1，y=2，交换失败！
    return 0;
}
```

2. 传址调用（正确，成功交换）

Code block

```
// 传址调用：形参是指针，接收实参的地址，直接操作原变量
void swap(int* a, int* b) {
    int temp = *a; // *a 解引用：取a指向的原变量x的值
    *a = *b;       // 修改a指向的原变量x的值
    *b = temp;     // 修改b指向的原变量y的值
}

int main() {
    int x = 1, y = 2;
    swap(&x, &y); // 传x、y的内存地址，传址调用
    // 结果：x=2，y=1，交换成功！
    return 0;
}
```

三、传址调用的核心特点

1. **直接操作原变量**：函数内通过 `*指针` 解引用，修改的是原变量的内存，而非副本，是修改函数外变量的核心手段。
2. **高效传递大对象**：传递数组、大结构体、类对象时，仅传指针（固定4/8字节），无需拷贝整个对象，大幅提升效率。
3. **数组传参的本质就是传址**：C++中数组作为函数参数时，会自动退化为指针，本质是传址调用，因此函数内修改数组元素会直接影响原数组（呼应你之前的数组传参问题）：

Code block

```
void modifyArr(int* arr, int size) {
    arr[0] = 100; // 等价于 *(arr+0) = 100，直接修改原数组
}
```

```
int main() {
    int arr[] = {1,2,3};
    modifyArr(arr, 3); // 数组退化为指针，传址调用
    // 原数组arr[0]变为100
    return 0;
}
```

四、传址调用 vs 传引用调用（易混淆点）

C++中还有「传引用调用（Call by Reference）」，两者**本质都是操作原变量**，但语法和细节有明显区别：

特性	传址调用（指针实现）	传引用调用（引用实现）
形参类型	指针（`int*`）	引用（`int&`）
调用语法	必须传地址（`&x`）	直接传变量（`x`）
函数内操作	需要解引用（`*a`）	直接用变量名（`a`）
空值风险	可能为空指针（`nullptr`）	不可能为空，必须绑定有效变量
本质	指针是独立变量，存储地址	引用是变量的别名，无独立内存
兼容性	C/C++ 通用	仅C++支持

传引用的交换函数（更简洁）

Code block

```
void swap(int& a, int& b) { // a是x的别名，b是y的别名
    int temp = a;
    a = b;
    b = temp; // 直接修改原变量x、y
}
int main() {
    int x=1, y=2;
    swap(x, y); // 直接传变量，无需&
    return 0;
}
```

五、传址调用的适用场景

1. **需要修改函数外的原变量**：如交换变量、修改数组/结构体、修改类对象状态。
2. **传递大对象**：如大数组、大结构体、自定义类，避免拷贝开销，提升性能。
3. **C语言兼容代码**：C语言无引用，只能用指针实现传址。
4. **只读大对象的安全传递**：用 `const` 修饰指针（如 `const int* arr`），既保证高效传址，又禁止函数内修改原数组（呼应你之前的 `const` 指针传数组问题）。

六、传址调用的避坑指南（新手高频踩坑）

1. 区分「指针本身」和「指针指向的内容」

传址调用中，**指针本身是传值的**：修改指针的指向（即指针变量本身的值），不会影响实参的指针；只有修改 `*指针`（指向的内容），才会影响原变量。

Code block

```
void func(int* p) {
    int b = 10;
    p = &b; // 仅修改形参p（指针副本）的指向，实参p完全不变！
    *p = 20; // 修改*p，会影响原变量
}
int main() {
    int x = 5;
    int* p = &x;
    func(p);
    // 结果：p仍指向x，x的值变为20
    return 0;
}
```

2. 必须保证指针指向有效内存

禁止传递空指针、野指针，否则解引用会触发**未定义行为**（程序崩溃、数据错乱等）。

3. 用 `const` 修饰只读指针

对于仅需读取、无需修改的大对象，用 `const` 修饰指针（如 `const int* arr`），从语法层面禁止函数内修改原数据，提升代码安全性。

七、总结

传址调用的核心口诀：**传地址，改原变量，用指针，高效安全**。

- 传值：改副本，不影响原变量，适合小对象只读。
- 传址：改原变量，高效传大对象，C/C++通用。
- 传引用：传别名，语法更简洁，C++专属，本质和传址一致。

要不要我给你整理一份**传值/传址/传引用的对比速查表**，或者补充几个传址调用的实战场景（比如修改结构体、传递vector指针）？

（注：文档部分内容可能由 AI 生成）